

DEVOPS: CI/CD

DAY 2 · DELIVERY, KUBERNETES, AND OPERATIONS



WHERE DAY 1 LEFT OFF

Day 1 built the first half of the pipeline. Every change to Shiplt is now built, tested, containerized, scanned, and blocked at the pull request if any of that fails. That known-good image is exactly where Day 2 begins.

The finished Day 1 pipeline

- A commit triggers CI: build and test on a clean runner
- The build is packaged as a container image, tagged with the commit SHA
- The image is pushed to Azure Container Registry
- Dependency, code, and secret scans gate the pull request, so nothing unsafe merges

The through-line:

Day 2 never rebuilds that image. It promotes the exact known-good artifact from Day 1 all the way to production.

DAY 2 AT A GLANCE

Day 2 takes that known-good image and delivers it: promoted with a human gate, run on a real orchestrator, provisioned as code, and monitored. Three modules, each closing a gap the last one exposed.

Today's three modules

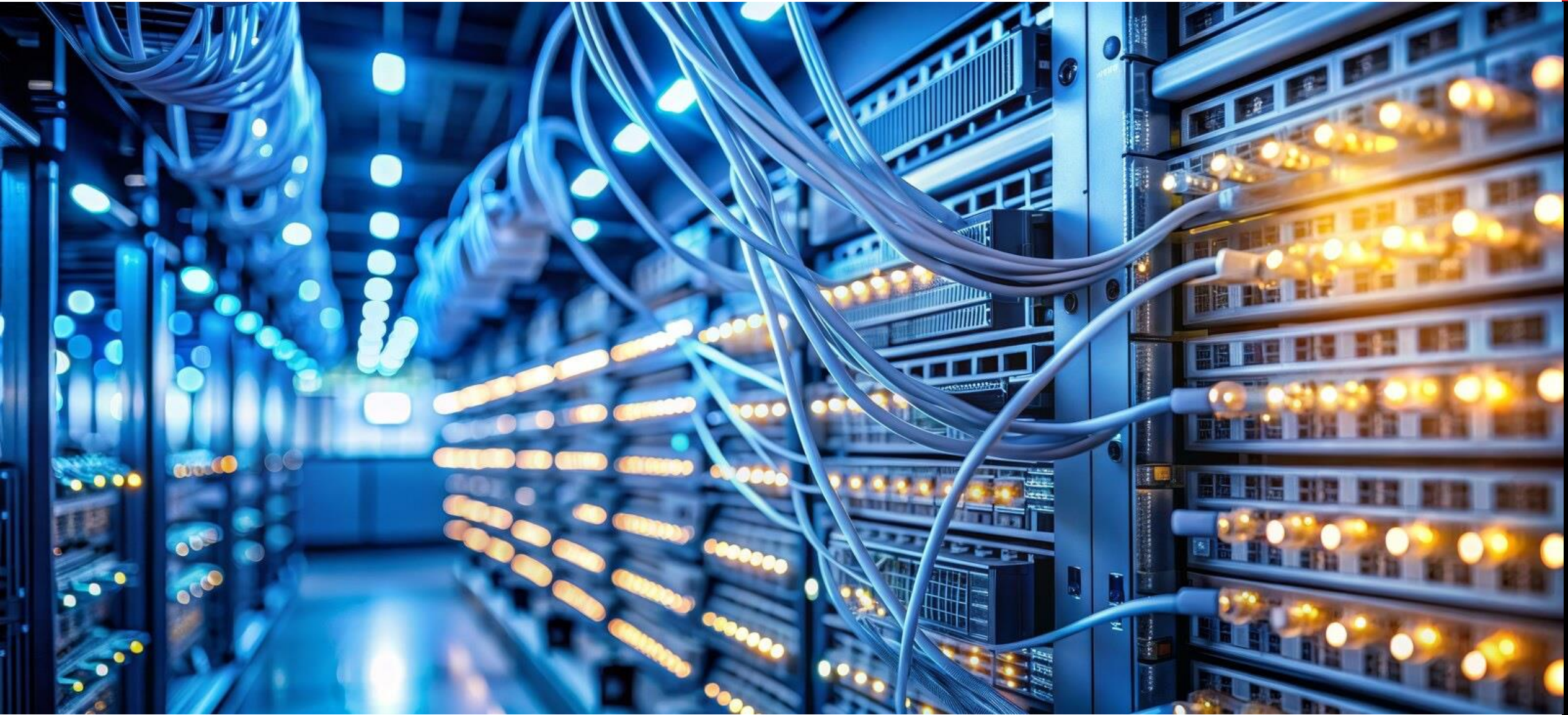
- Module 5: CD pipelines, promotion with approval, OIDC, and rollback
- Module 6: running Shiplt on Kubernetes and OpenShift with Helm
- Module 7: infrastructure as code with Bicep, and monitoring that closes the DORA loop

Where we land:

By the end of today, merging a change ships it to a monitored production deployment on its own.

CD PIPELINES AND DEPLOYMENT STRATEGIES

FROM BUILT TO DELIVERED



LEARNING OBJECTIVES

Day 1 stopped at an image in a registry, which is potential energy, not delivered software. Continuous delivery, CD, is the discipline of getting that exact image into environments safely and repeatably, with a human in the loop exactly where the business needs one and nowhere else.

- Distinguish continuous delivery from continuous deployment, and choose between them
- Promote one build through dev, staging, and production using GitHub Environments and required reviewers
- Authenticate from GitHub Actions to Azure with OIDC federation instead of a stored secret
- Compare rolling and blue/green deployments and say when each one fits
- Add health checks that trigger an automatic rollback when a deploy goes bad

DELIVERY VS DEPLOYMENT, AND WHY IT MATTERS

The two terms differ by exactly one thing: whether a human approves the last step. That one difference is a real business decision, not a technicality, and getting it wrong either slows you down for no reason or ships something nobody was ready for.

One word, two disciplines

- Continuous delivery: every change is always deployable, and a human clicks go for production
- Continuous deployment: every change that passes the pipeline ships to production on its own
- Both automate everything up to that final step; only the last click differs
- Full automation is earned: it needs strong tests, fast rollback, and real monitoring first

Key Point:

Continuous deployment is not a more advanced trophy to chase. It is a choice you make only once you trust your safety net.



CHOOSING DELIVERY OR DEPLOYMENT

Neither choice is more advanced than the other. The right one depends on your risk tolerance, your regulatory reality, and how much you trust your safety net. Most real teams end up mixing the two.

How to decide

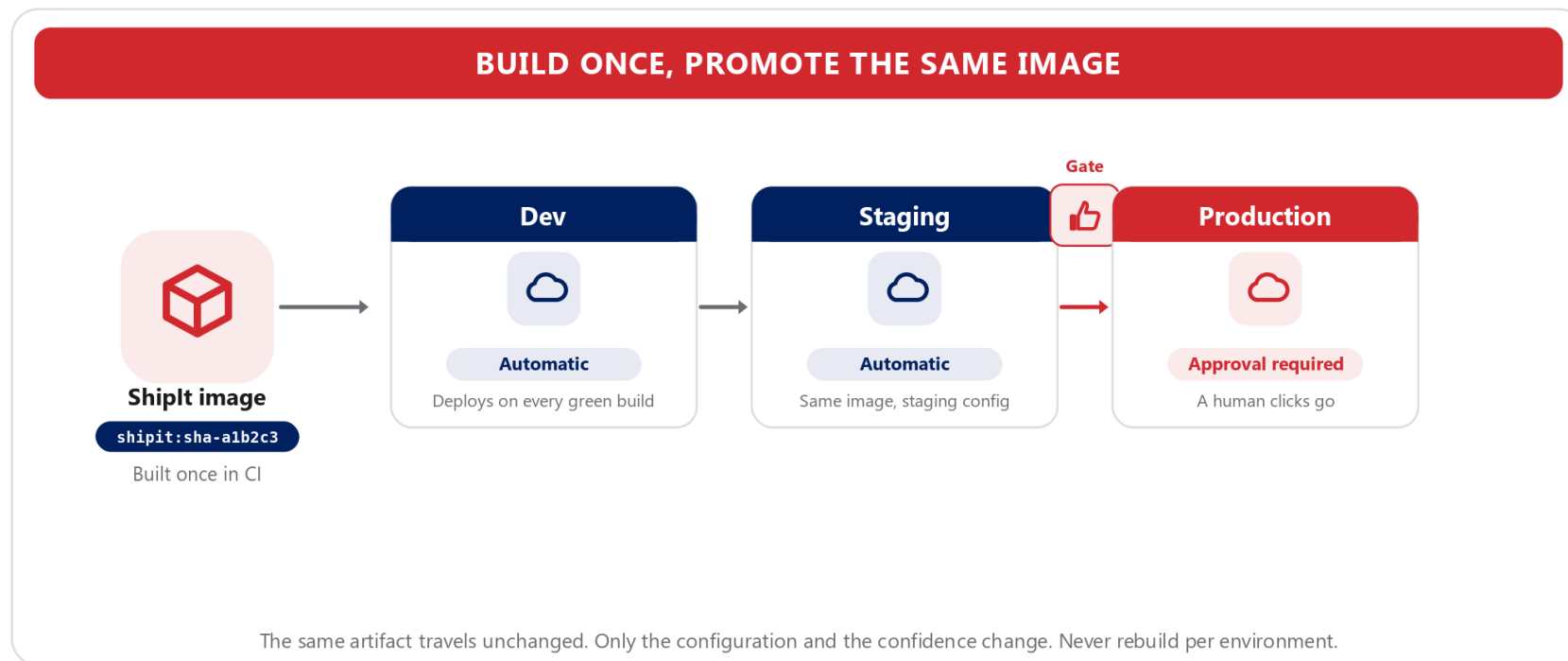
- High blast radius or regulated change: lean delivery, keep the human gate
- Strong test coverage plus instant rollback: deployment becomes safe
- Mix per environment: automatic to staging, approval before production
- Tie the choice to a DORA target: are you protecting stability or speeding lead time?

Best practice:

The common, sensible pattern: deploy automatically to staging on every green build, and require one approval before production.

THE PROMOTION PATH

The same image moves through environments unchanged. What changes is the configuration and your confidence. Seeing that one artifact travels, and is never rebuilt, is the key mental model for the whole module.



Build once in CI, then promote that exact image through dev and staging automatically, with a human gate before production.

GITHUB ENVIRONMENTS



An environment in GitHub is more than a label. It carries its own protection rules, secrets, and approval requirements, which is exactly what lets one pipeline behave differently at each stage without a separate workflow per environment.

What an environment gives you

- Environment-scoped secrets and variables, so staging and prod hold different values
- Required reviewers, the approval gate that pauses the run before it proceeds
- Wait timers, a forced cool-down before a deploy continues
- Deployment branch rules, so only main can deploy to production

Key Point:

The approval gate is a setting on the environment, not code in your workflow. Attaching a job to the environment is what turns the rule on.

ATTACHING JOBS TO ENVIRONMENTS

Attaching a job to an environment is what activates that environment's protection rules. The approval gate is one key plus some settings in the GitHub UI, not a pile of workflow logic.

```
jobs:
  deploy-staging:
    needs: build                # get the SHA the build produced
    runs-on: ubuntu-latest
    environment: staging        # staging rules + secrets apply
    steps:
      - run: ./deploy.sh staging ${ needs.build.outputs.sha }

  deploy-production:
    needs: [build, deploy-staging] # build for its output, staging for order
    runs-on: ubuntu-latest
    environment: production        # required reviewer pauses here
    steps:
      - run: ./deploy.sh production ${ needs.build.outputs.sha }
```

The environment: key is the whole trick. Because production has a required reviewer, the run pauses here until a human approves.

WHY STORED CLOUD CREDENTIALS ARE A LIABILITY

The classic way to let a pipeline touch Azure is a long-lived secret stored in the repo. That secret is a standing risk: it can leak, it rarely gets rotated, and it keeps working long after it should have been revoked.

The problem with a stored secret

- A long-lived secret can leak in a log, a fork, or a screen share
- Rotation is manual, so in practice it rarely happens
- If it is exposed, the blast radius is everything it can reach, for as long as it lives
- It outlives the person and the project that created it

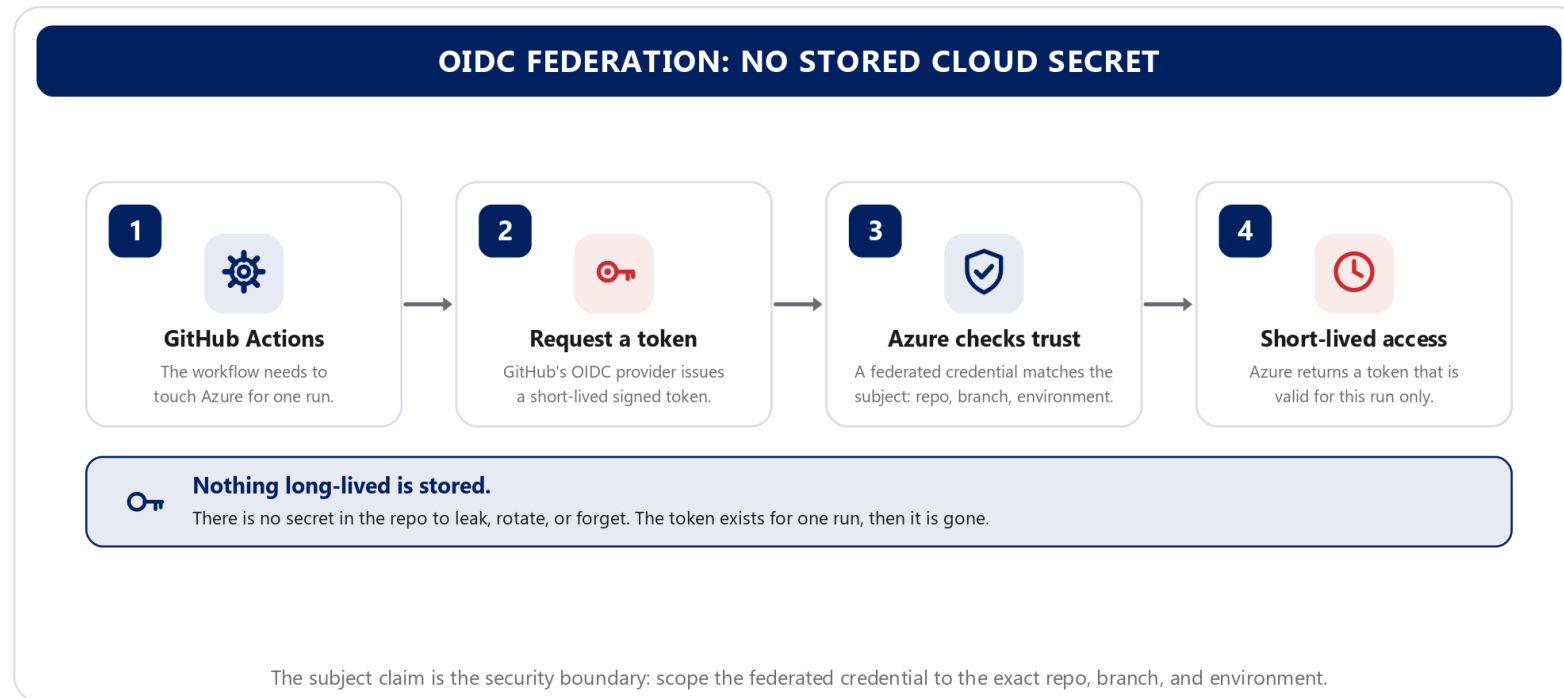
Watch out:

A secret you never have to store is a secret that can never leak. That is the whole idea behind OIDC federation.



HOW OIDC FEDERATION WORKS

OIDC, OpenID Connect, lets GitHub prove its identity to Azure for a single run and receive a short-lived token, so there is no secret to store or leak. Understanding the four-step handshake makes the setup make sense instead of feeling like magic.



The workflow requests a token, GitHub signs it, Azure trusts it through a federated credential scoped to the repo and environment, and returns short-lived access.

CONFIGURING OIDC TO AZURE

The setup is a small permissions block in the workflow plus a federated credential on the Azure side. Notice what is missing: there is no client secret anywhere in the file.

```
permissions:
  id-token: write      # allow the workflow to request an OIDC token
  contents: read

jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: azure/login@v3      # no client-secret input
        with:
          client-id: ${vars.AZURE_CLIENT_ID}
          tenant-id: ${vars.AZURE_TENANT_ID}
          subscription-id: ${vars.AZURE_SUBSCRIPTION_ID}
```

The federated credential on the Azure app is scoped to subject repo:org/shipit:environment:production. That subject is the security boundary.

POP QUIZ: DELIVERY VS DEPLOYMENT

A bank ships to production only after a change-approval board signs off, but everything up to prod is fully automated. Is that continuous delivery or continuous deployment, and what GitHub feature implements the gate?



3 MINUTES

POP QUIZ: DELIVERY VS DEPLOYMENT

A bank ships to production only after a change-approval board signs off, but everything up to prod is fully automated. Is that continuous delivery or continuous deployment, and what GitHub feature implements the gate?

Answer:

Continuous delivery: automated and always ready to ship, with a human gate before production.

- The board sign-off is the human approval, so it is delivery, not deployment
- The gate is a GitHub Environment named production with a required reviewer
- Everything up to that gate stays fully automated, which is what keeps it fast



ROLLING UPDATES

The default way to update a running service is to replace its instances a few at a time, so the service never fully goes down. Knowing how it behaves tells you exactly what can go wrong during a release.

How a rolling update behaves

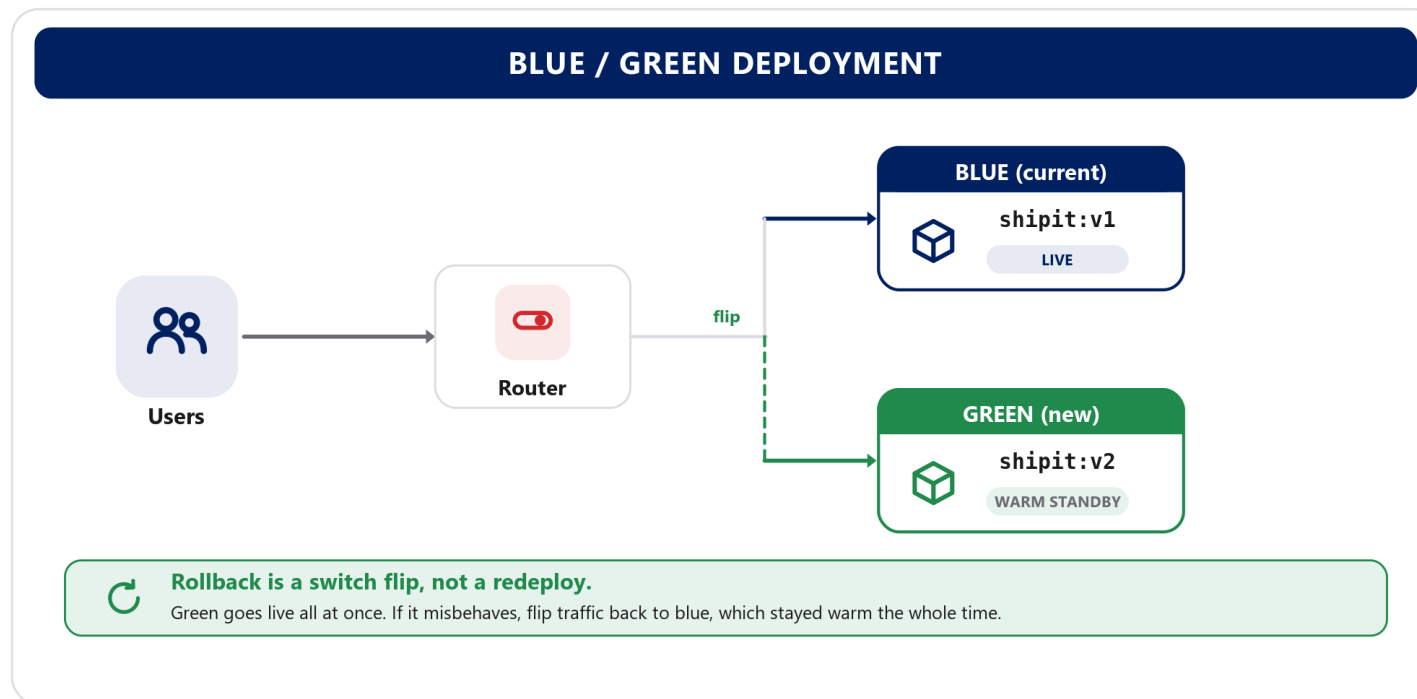
- Replace instances gradually, a few at a time, never all at once
- Old and new versions run side by side for a short window
- The change must be backward compatible during that overlap
- Rolling all the way back is slow, because it is another gradual pass

Watch out:

Because old and new run together briefly, a rolling update assumes both versions can coexist, especially against the same database.

BLUE/GREEN DEPLOYMENT

When you cannot tolerate mixed versions, or you want instant rollback, you stand the new version up beside the old one and switch all traffic at once. Nothing is gradual, so there is no overlap window to reason about.



Blue runs live while green runs the new version beside it. Flip all traffic to green at once, and keep blue warm so rollback is just flipping back.

CHOOSING A STRATEGY

The choice trades cost against safety and rollback speed. There is no universally right answer, only the right answer for this change and this budget.

Rolling

- Cheap: no second copy of the app
- Gradual cutover, no full outage
- Needs backward-compatible changes
- Rollback is slow, another gradual pass
- Good default for most services

Blue/Green

- Instant rollback: flip back to blue
- Clean cutover, no mixed versions
- Costs double resources briefly
- Great for risky or hard-to-reverse releases
- Canary is the next step beyond this course

HEALTH CHECKS AS THE DEPLOY'S SAFETY NET



An automated deploy is only safe if the pipeline can tell whether the new version is actually healthy. That is why ShiptIt exposes two endpoints, and why we built them on Day 1: they are the signal the pipeline watches after every deploy.

What the pipeline watches

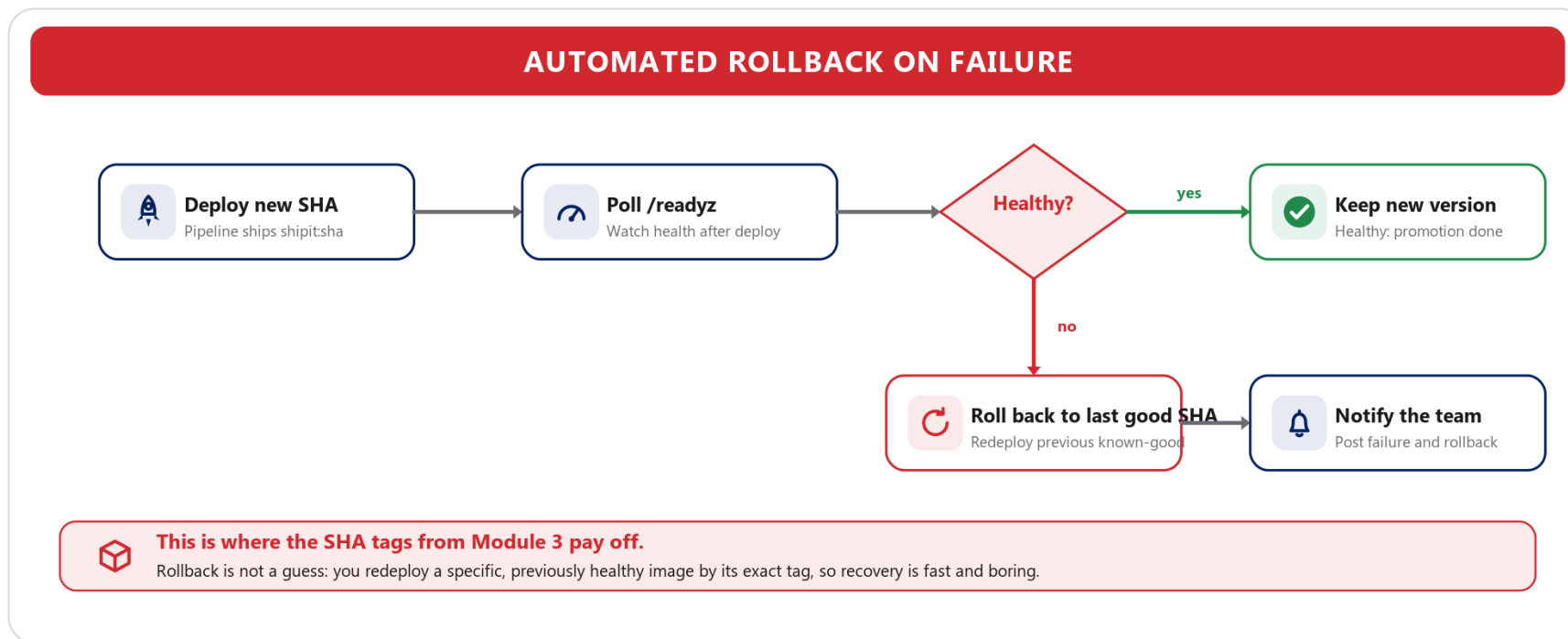
- `/healthz` answers is the process alive and running
- `/readyz` answers is it actually ready to serve real traffic
- After deploy, the pipeline polls readiness instead of assuming success
- A failed check is the trigger for rollback, not a human noticing later

Key Point:

Deploying and hoping is not a strategy. The pipeline must verify health itself, or the rollback can never be automatic.

AUTOMATED ROLLBACK ON FAILURE

Rollback should be boring and automatic, because the worst possible time to improvise is in the middle of a bad deploy at the worst possible hour. The pipeline decides, not a panicked human.



Deploy, poll health, and branch: healthy keeps the new version; unhealthy rolls back to the previous known-good SHA and notifies the team.

POP QUIZ: ROLLBACK DESIGN

Your rolling update is halfway done and the new pods are failing their readiness check. What should a well-built pipeline do next, and why would blue/green handle this moment differently?



4 MINUTES

POP QUIZ: ROLLBACK DESIGN

Your rolling update is halfway done and the new pods are failing their readiness check. What should a well-built pipeline do next, and why would blue/green handle this moment differently?

Answer:

It stops promoting and automatically rolls back to the last known-good SHA; with blue/green, traffic never moved to green, so rollback is just not flipping the switch.

- Rolling: halt the rollout and redeploy the previous known-good image automatically
- The failing readiness check is the trigger, no human has to catch it
- Blue/green: green failed before any traffic moved, so blue simply keeps serving



SECRETS MANAGEMENT ACROSS ENVIRONMENTS

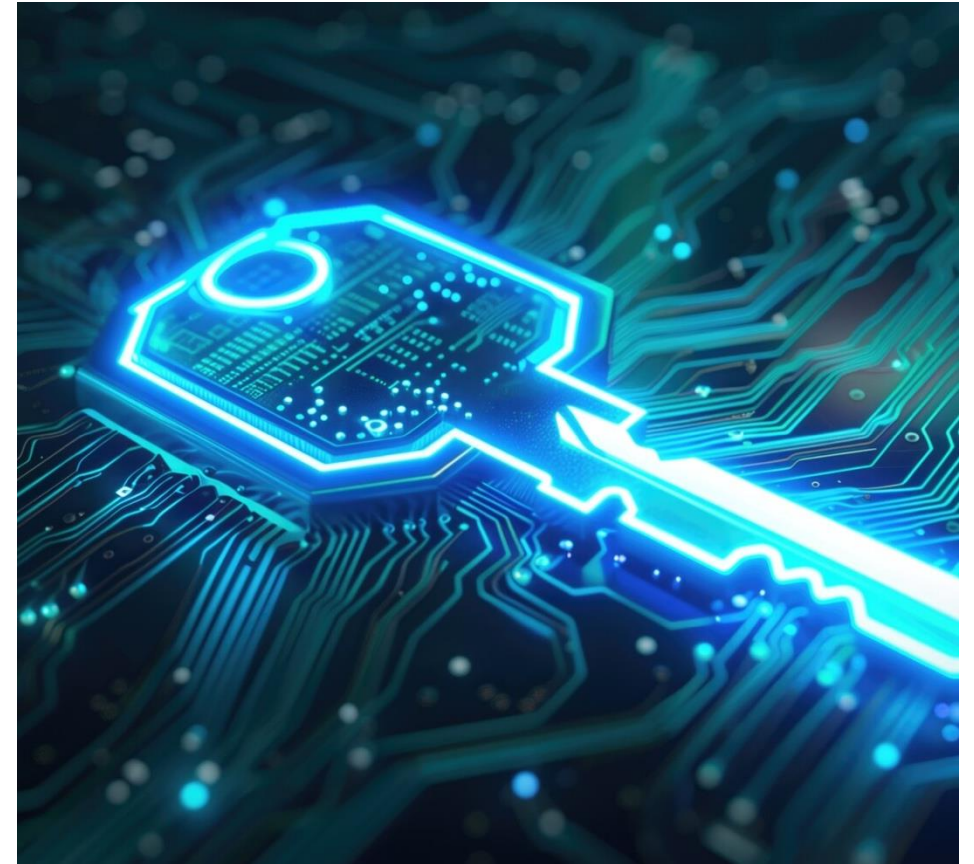
Different environments need different secrets, and the pipeline should pull the right ones at the right stage without any of them living in your code or your image. Where a secret lives matters as much as what it protects.

A layered approach

- GitHub environment secrets hold per-stage values, staging and prod kept separate
- OIDC handles cloud access, so there is no static Azure secret at all
- Application secrets come from a store like Azure Key Vault at runtime
- Never bake a secret into the image; the image is shared across every environment

Watch out:

The image is identical in every environment, so anything baked into it is exposed everywhere. Secrets belong outside the artifact, injected per environment.



TRACEABILITY: LINKING DEPLOYS TO WORK ITEMS



When something breaks in production, the first question is always what changed, and traceability answers it in seconds instead of an afternoon of archaeology. It is also what auditors and incident responders both quietly depend on.

Wiring the trail together

- Reference work items in commits and pull requests, for example Fixes #142
- The deploy records which commits and items it carried to each environment
- A production issue links back to the exact change and the person who made it
- Auditors get history for free; responders get answers fast

Key Point:

The commit, the pull request, the work item, and the deploy should form one unbroken chain. That chain is what turns a 2am incident into a five-minute answer.

CD PITFALLS TO AVOID

A handful of mistakes turn continuous delivery from a safety net into a foot-gun. Naming them now means you can design around them instead of discovering each one in production.

The usual traps

- Rebuilding per environment, which lets staging and prod quietly drift apart
- So many approvals that people rubber-stamp them without looking
- A rollback path that was never tested, so it fails the one time you need it
- Config or secrets baked into the image instead of injected per environment

Best practice:

An untested rollback is not a safety net. Practice the rollback deliberately, the same way you'd test a smoke alarm.

KEY TAKEAWAYS

Continuous delivery is where the pipeline stops building and starts shipping, safely and on purpose. The same image you built on Day 1 now travels to production behind a human gate and a health check.

- Delivery keeps a human gate before production; deployment does not. It's a risk choice, not a maturity badge
- Build once and promote the same image; never rebuild per environment
- OIDC federation replaces stored cloud secrets with a token that lives for one run
- Rolling trades rollback speed for cost; blue/green trades cost for instant rollback
- Health checks are what make automated rollback to a known-good SHA possible

LAB 5: BUILD A CD PIPELINE WITH APPROVAL AND ROLLBACK

Now build a real CD pipeline for Shipt. You will authenticate to Azure with OIDC, promote a build to staging automatically, gate production behind an approval, and make a failed health check roll the deploy back on its own.

In this lab you will

- Configure OIDC so the workflow reaches Azure with no stored secret
- Create staging and production environments, with a required reviewer on production
- Add a post-deploy health check that rolls back to the last good SHA on failure

Hands-on lab:

About 45 to 60 minutes. Done when: production won't deploy without approval, and a bad deploy rolls itself back automatically.



COMING UP NEXT

You can now promote and roll back safely, but we have been hand-waving about where the app actually runs. A single container on a single host still has an obvious failure mode: the host dies, and so does your app.

Where we go from here

- Lab 5: a gated CD pipeline that promotes on approval and rolls back on failure
- Module 6: Kubernetes, the orchestrator that runs and heals these containers
- We deploy Shiplt to AKS with Helm, on both AKS and Red Hat OpenShift

DEPLOYING TO KUBERNETES AND OPENSIFT

WHERE DEPLOY BECOMES REAL



LEARNING OBJECTIVES

Module 5's pipeline deploys somewhere, but production containers rarely run alone on one host. Kubernetes is the orchestrator that keeps the right number of healthy copies running, restarts the ones that die, and routes traffic to the healthy ones. This is where deploy becomes real.

- Explain pods, deployments, services, and ingress, and how a request reaches a container
- Package and deploy Shiplt with a Helm chart
- Deploy from GitHub Actions with Helm (helm upgrade) to AKS
- Externalize configuration with ConfigMaps and Secrets
- Use readiness probes and scaling, and troubleshoot with kubectl
- Describe how OpenShift differs from upstream Kubernetes

THE PROBLEM KUBERNETES SOLVES

One container on one machine has an obvious failure mode: the machine dies, and so does your app. Kubernetes, often shortened to K8s, exists to keep a declared number of healthy copies running across many machines without a human watching.

What an orchestrator does for you

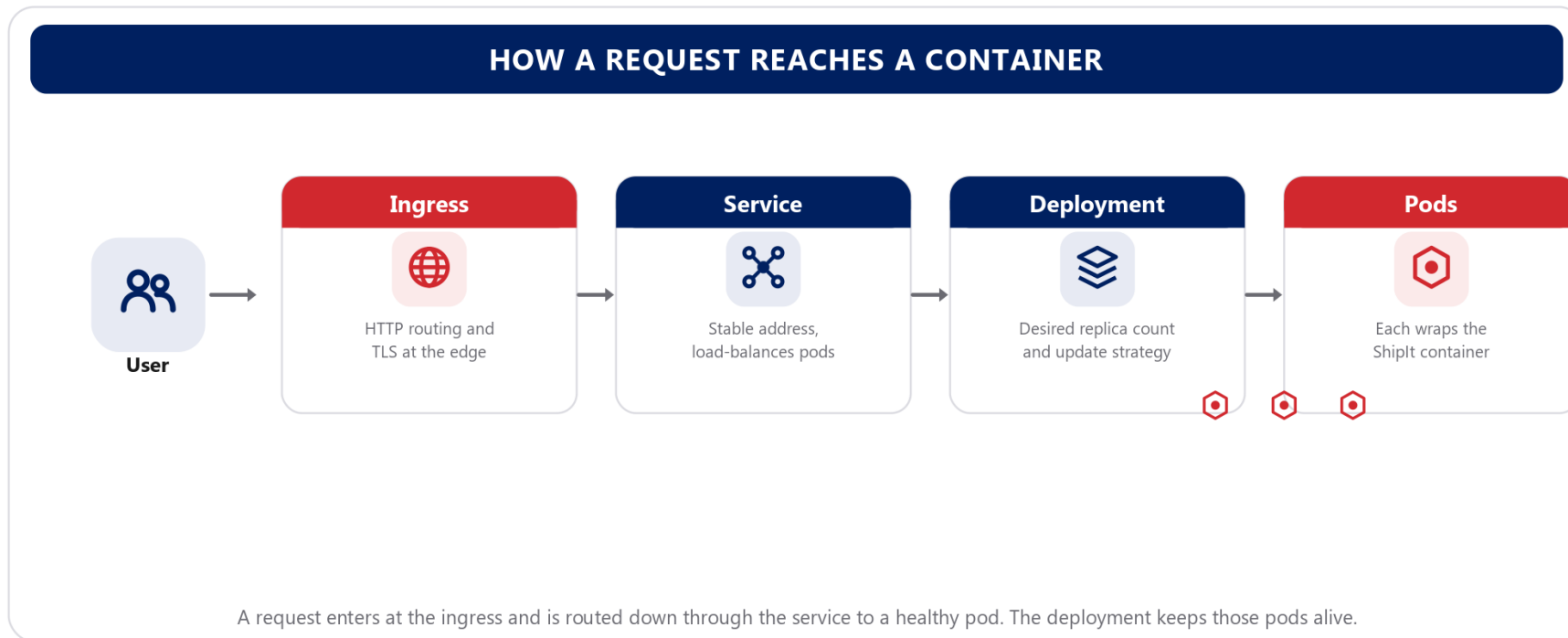
- Self-healing: a pod that dies is restarted or rescheduled automatically
- Scaling: run more copies on demand, across many machines
- Scheduling: it decides which machine each copy runs on
- Declarative desired state: you say what you want, it figures out how

Key Point:

You describe the end state you want, for example run three healthy copies of Shipt, and Kubernetes continuously works to make reality match that.

PODS, DEPLOYMENTS, SERVICES, AND INGRESS

Four objects carry most of what you deploy, and knowing how they stack explains how a user request actually reaches your container. Read it left to right: the request enters at the edge and is routed down to a healthy pod.



Ingress routes HTTP in, a service gives a stable address and load-balances, a deployment keeps the pods alive, and each pod wraps the Shiplt container.

PODS AND DEPLOYMENTS

You almost never create a pod directly. You declare a deployment, and it keeps the pods matching what you asked for. That reconciliation is the whole self-healing story in one object.

How the two relate

- A pod is one or more containers that share a network address
- A deployment declares the desired replica count and the update strategy
- The controller constantly reconciles reality to the spec
- Kill a pod and the deployment makes a new one, that is self-healing

Key Point:

Delete a pod by hand and it comes back. You are not managing pods, you are managing the desired state a deployment enforces.

SERVICES AND INGRESS

Pods come and go and their IP addresses change with them, so you need a stable front door. A service provides that inside the cluster; an ingress exposes it to the outside world with HTTP routing and TLS.

The stable front door

- A service is a stable address that load-balances across the live pods
- ClusterIP is internal only; LoadBalancer exposes it outside the cluster
- An ingress adds HTTP routing and TLS termination at the edge
- That is how Shiplt's status page gets reached from a browser

Key Point:

A service solves the moving-target problem: clients talk to one stable name, and Kubernetes routes to whichever pods are currently healthy.

A MINIMAL DEPLOYMENT AND SERVICE

Seeing raw Kubernetes YAML once, before Helm hides it, makes clear what Helm is actually generating for you. This is the deployment and service for Shipt, by SHA.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: shipit }
spec:
  replicas: 3
  selector: { matchLabels: { app: shipit } }
  template:
    metadata: { labels: { app: shipit } }
    spec:
      containers:
        - name: shipit
          image: shipitacr.azurecr.io/shipit:sha-a1b2c3
          ports: [ { containerPort: 8080 } ]
---
apiVersion: v1
kind: Service
metadata: { name: shipit }
spec:
  selector: { app: shipit }
  ports: [ { port: 80, targetPort: 8080 } ]
```

The service selects pods by the label app: shipit, so it always points at the current, healthy pods no matter how often they restart.

WHY RAW YAML DOESN'T SCALE ACROSS ENVIRONMENTS

The same manifest needs different values in dev, staging, and prod. Copy-pasting three near-identical files is exactly how drift and mistakes creep in, the same problem we fought in Module 5, one layer down.

The trouble with copies

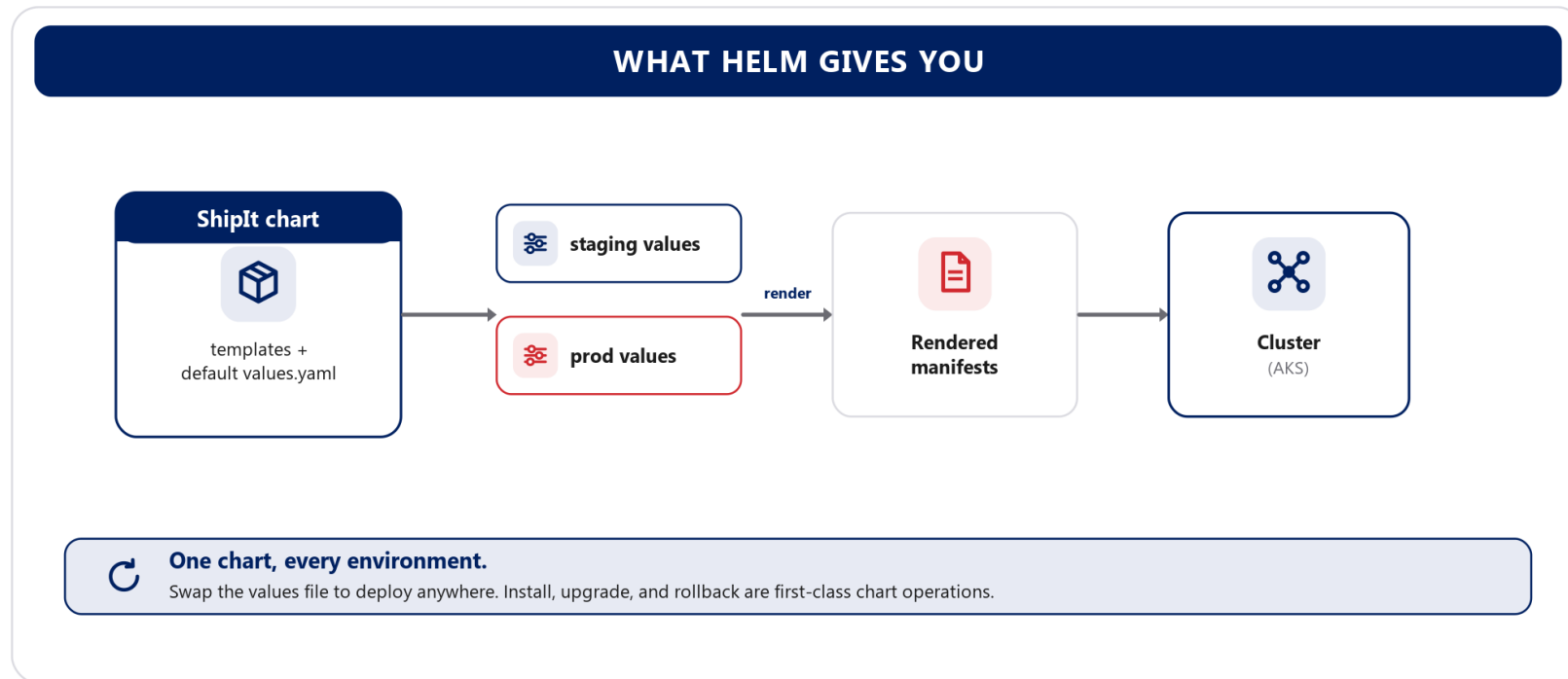
- Duplicated manifests drift apart, one gets edited and the others don't
- Environment-specific values end up scattered across files
- There is no single versioned unit you can install or roll back
- A one-line prod difference is easy to miss in a wall of YAML

Watch out:

You already know this smell from Module 5: rebuilding or re-editing per environment is how staging and prod quietly stop matching.

WHAT HELM GIVES YOU

Helm packages a set of manifests into a versioned chart with a values file, so one chart deploys to every environment by swapping values. It turns a pile of YAML into a single, versioned unit you can install, upgrade, and roll back.



One chart of templates plus a per-environment values file renders into manifests for the cluster. Install, upgrade, and rollback become first-class operations.

THE SHIPIT HELM CHART

Now the deployment and service become templates, and the image tag and config become values. This is exactly what the pipeline needs to promote one chart across environments by passing in the SHA.

```
# values.yaml (defaults; overridden per environment)
image:
  repo: shipitacr.azurecr.io/shipit
  tag: sha-a1b2c3          # the pipeline passes this in
replicas: 3
region: eastus

# templates/deployment.yaml (excerpt)
spec:
  replicas: {{ .Values.replicas }}
  containers:
    - name: shipit
      image: "{{ .Values.image.repo }}:{{ .Values.image.tag }}"
```

The pipeline runs helm upgrade and passes the image tag as a value, so the chart version and the running image version never drift apart.

CONFIGMAPS AND SECRETS

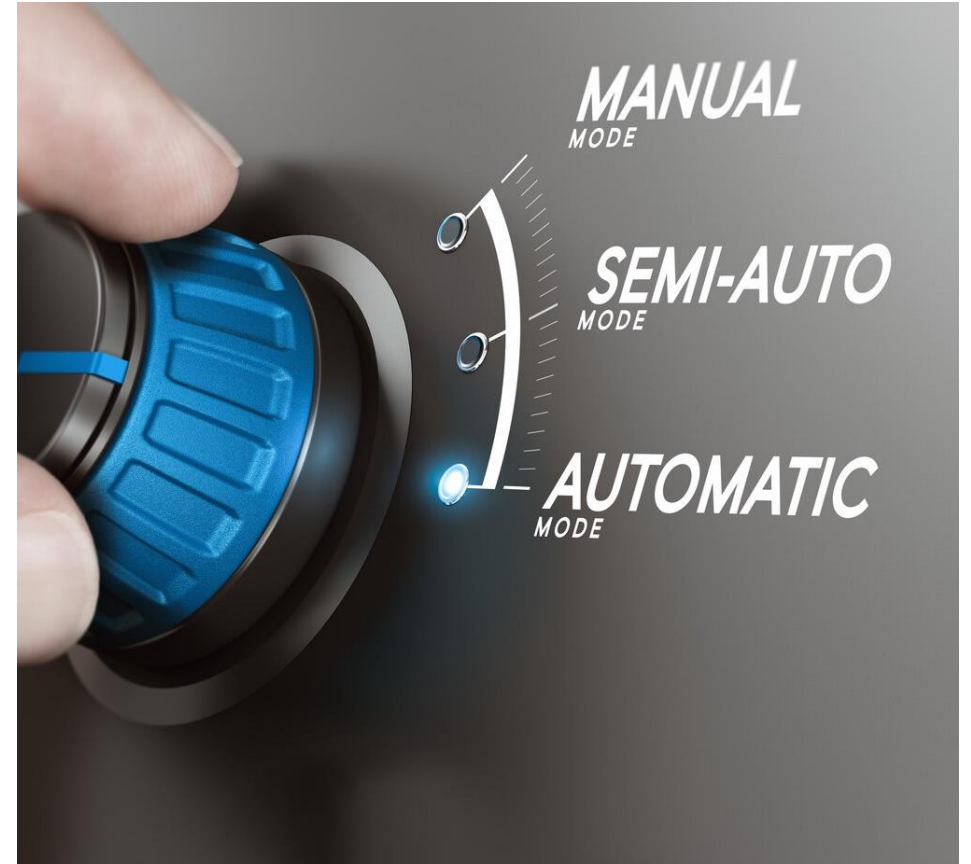
The config-by-environment choice we made in Module 3 pays off here: Shiplt reads its region and banner color from the environment, so a ConfigMap can change its behavior with no rebuild. Sensitive values go in a Secret instead.

Externalizing configuration

- ConfigMap holds non-sensitive config, mounted as env vars or files
- Secret holds sensitive values, referenced the same way
- Change config per environment without touching the image
- Kubernetes Secrets are base64, not encrypted at rest by default

Key Point:

Same image everywhere, different ConfigMap per environment. That is build-once-configure-per-environment made real in the cluster.



POP QUIZ: OBJECT ROLES

A pod's IP keeps changing as it restarts.
What object gives clients a stable way to
reach it, and what actually keeps the
right number of pods running?



3 MINUTES

POP QUIZ: OBJECT ROLES

A pod's IP keeps changing as it restarts. What object gives clients a stable way to reach it, and what actually keeps the right number of pods running?

Answer:

A Service gives the stable address; a Deployment keeps the replica count healthy.

- Service: one stable name that load-balances across the current healthy pods
- Deployment: reconciles the pod count to the number you declared
- Different jobs: how you reach it versus how it stays alive



READINESS AND LIVENESS PROBES

Kubernetes cannot know your app is ready to serve just because the process started. Probes are how you tell it, and they are what make rolling updates and the Module 5 health checks trustworthy. Shiplt's two endpoints map straight onto them.

Two probes, two questions

- Liveness: is the process alive? If it fails, restart the pod
- Readiness: is it ready to serve? If it fails, hold traffic back
- Shiplt maps /healthz to liveness and /readyz to readiness
- Readiness is what gates a rolling update, no traffic until ready

Key Point:

A pod can be alive but not ready, still warming up or waiting on a dependency. Readiness is what keeps users off it until it can actually serve.

ROLLING UPDATES AND SCALING IN KUBERNETES

The rolling strategy we discussed in Module 5 is native here: the deployment does it for you, gated by readiness. And scaling, up or down, is one field or one command.

Updates and scaling, built in

- maxSurge and maxUnavailable control how fast pods are swapped
- Readiness gates the rollout, a pod gets traffic only once it is ready
- Scale by changing the replica count: kubectl scale or edit the value
- A bad version stalls the rollout safely instead of taking everything down

Key Point:

Because readiness gates the rollout, a broken new version simply never becomes ready, so the update stalls with the old version still serving.

DEPLOYING FROM GITHUB ACTIONS

This replaces Module 5's placeholder target with a real cluster deploy, closing the loop from commit to running pods. Same OIDC login, same SHA-tagged image, now landing on AKS.

```
- uses: azure/login@v3          # OIDC, no stored secret
  with:
    client-id: ${vars.AZURE_CLIENT_ID}
    tenant-id: ${vars.AZURE_TENANT_ID}
    subscription-id: ${vars.AZURE_SUBSCRIPTION_ID}
- uses: azure/aks-set-context@v5
  with:
    resource-group: shipit-rg
    cluster-name: shipit-aks
- run: |
  helm upgrade --install shipit charts/shipit \
    --set image.tag=${github.sha} --wait
```

The same known-good image from Day 1 now lands on AKS, gated by the same Module 5 approval. --wait makes the job fail if the rollout never becomes ready.

TROUBLESHOOTING WITH KUBECTL



Deployments fail in a handful of predictable ways, and a short set of kubectl commands diagnoses almost all of them. This is a skill students use on their first day on the job, so it is worth drilling.

Your first four commands

- kubectl get pods: status at a glance, are they Running or not
- kubectl describe pod: events explaining why it will not start
- kubectl logs (add --previous): the app's own output, including a crash
- Read the status: ImagePullBackOff vs CrashLoopBackOff vs Pending

Best practice:

The pod status usually names the problem: ImagePullBackOff is a pull or auth issue, CrashLoopBackOff is the app dying on start, Pending is nowhere to schedule it.

POP QUIZ: TROUBLESHOOTING

A pod is stuck in CrashLoopBackOff.
Which two kubectl commands do you
run first, and what are you looking for in
each?



4 MINUTES

POP QUIZ: TROUBLESHOOTING

A pod is stuck in CrashLoopBackOff. Which two kubectl commands do you run first, and what are you looking for in each?

Answer:

kubectl describe pod for the events and restart reason, and kubectl logs --previous for why the last run crashed.

- describe pod: events, probe failures, and the restart reason
- logs --previous: the output from the run that just died, not the new one
- Together they separate a scheduling or config problem from an app crash



RED HAT OPENSIFT: SAME CORE, DIFFERENT EDGES

Many enterprises run Red Hat OpenShift rather than plain Kubernetes, so it is worth knowing what transfers directly and what is different. The good news: it is Kubernetes underneath, so almost everything you just learned still applies.

What is the same, what is different

- Underneath it is Kubernetes, your objects and kubectl still work
- Projects are namespaces with extra guardrails built in
- Routes do what ingress does, exposing HTTP outside the cluster
- The oc CLI sits alongside kubectl, with extra OpenShift verbs

Key Point:

You do not relearn Kubernetes for OpenShift. You learn a handful of different names and one stricter security posture, covered next.

AKS VS OPENSIFT AT A GLANCE

A side-by-side keeps the differences concrete so you can move between the two without second-guessing. The shape is identical; only the edges differ.

AKS VS OPENSIFT AT A GLANCE		
	AKS	OpenShift
Edge routing	 Ingress	 Route
Tenancy unit	 Namespace	 Project (namespace + guardrails)
Security defaults	 Permissive	 Strict: no root, SCCs
CLI	 kubectl	 oc plus kubectl
Platform extras	 Self-managed add-ons	 Included in the platform
 Same Kubernetes underneath. The Shiplt Helm chart runs on both with small tweaks: a route instead of an ingress, and an image that already runs as non-root.		

Ingress becomes a route, a namespace becomes a project, security defaults get stricter, and oc joins kubectl. The Shiplt chart runs on both with small tweaks.

WHY OPENSIFT'S STRICT DEFAULTS MATTER FOR OUR IMAGE

This is where the Module 3 decision to run Shiplt as a non-root user pays off. OpenShift refuses to run a container that demands root, so a well-built image just works, while a sloppy one fails on arrival.

Strict by default, and that is good

- Security context constraints, SCCs, limit what a container may do
- The big one: containers may not run as root by default
- Images that assume root break immediately on OpenShift
- Shiplt's multi-stage image already runs as non-root, so it just runs

Key Point:

The habits that make an image secure are the same habits that make it portable. Our non-root image runs unchanged on the stricter platform.

POP QUIZ: CONFIG CHANGE

You need staging to show a yellow banner and production a green one, from the same image. What Kubernetes object do you change, and do you rebuild the image?



3 MINUTES

POP QUIZ: CONFIG CHANGE

You need staging to show a yellow banner and production a green one, from the same image. What Kubernetes object do you change, and do you rebuild the image?

Answer:

Change a ConfigMap (or the Helm value that sets it) per environment. No rebuild, because Shiplt reads the banner color from its environment.

- The image is identical in both environments; only the ConfigMap differs
- In Helm terms, it is one value in the per-environment values file
- No rebuild means no new image, no re-scan, no drift



KEY TAKEAWAYS

Kubernetes is the orchestrator that keeps Shiplt running and healthy, and Helm is how we package one deploy unit for every environment. The image from Day 1 now runs on a real cluster, gated by the Module 5 approval.

- Kubernetes keeps a declared number of healthy pods running and self-heals
- Ingress to service to deployment to pods is how a request reaches a container
- Helm packages one versioned chart that deploys to every environment by values
- ConfigMaps and Secrets externalize config; probes make rollouts safe
- OpenShift is Kubernetes with routes, projects, and stricter, non-root security

LAB 6: DEPLOY SHIPT TO AKS WITH HELM

Now put ShiptIt on a real cluster from the pipeline. You will write a Helm chart, deploy it to AKS through the Module 5 approval gate, change config with no rebuild, and deliberately break a pod so you can practice diagnosing it.

In this lab you will

- Write a Helm chart for ShiptIt with probes and a ConfigMap, and deploy it with helm upgrade
- Replace Lab 5's placeholder deploy with a real AKS deploy from the pipeline
- Change the banner via config with no rebuild, then break a pod and diagnose it with kubectl

Hands-on lab:

About 60 minutes. Done when: a merge to main deploys ShiptIt to AKS through the gate, and you can name why a broken pod failed.



COMING UP NEXT

Shiplt now runs on a real cluster, promoted by the pipeline. But we clicked through the Azure portal to create that cluster and its registry, which means nobody can recreate them reliably or review the change like code.

Where we go from here

- Lab 6: Shiplt deployed to AKS with Helm, through the approval gate
- Module 7: define that infrastructure as code with Bicep, deployed by the pipeline
- Then instrument Shiplt and watch it, closing the loop back to the DORA metrics

INFRASTRUCTURE AS CODE AND MONITORING

CLOSE THE LOOP



LEARNING OBJECTIVES

Two gaps remain. Every Azure resource so far was created by hand, so it cannot be recreated or reviewed like code. And we deploy without watching. Infrastructure as code, IaC, fixes the first; monitoring fixes the second and closes the DevOps loop we opened on Day 1.

- Explain IaC principles: desired state, idempotence, and repeatability
- Write Bicep to provision an Azure resource, using parameters and modules
- Deploy Bicep from GitHub Actions with OIDC
- Instrument Shiplt with Application Insights and collect telemetry
- Build a dashboard and an alert, and correlate a deploy with a performance change
- Use observability data to measure the DORA metrics from Module 1

THE PROBLEM WITH CLICK-OPS

Every resource we made by hand in the portal is a resource nobody can recreate exactly, review, or roll back, and it drifts the moment someone tweaks a setting. It is the same works-on-my-machine problem, one layer down in the stack.

Why manual infrastructure hurts

- Unrepeatable: you cannot stand the same environment up again with confidence
- No review or history: a portal click leaves no diff and no reviewer
- Configuration drift: someone changes a setting and nothing records it
- Disaster recovery becomes guesswork under pressure

Watch out:

If you cannot recreate an environment from a file in source control, you do not really know what is in it.

IAC PRINCIPLES

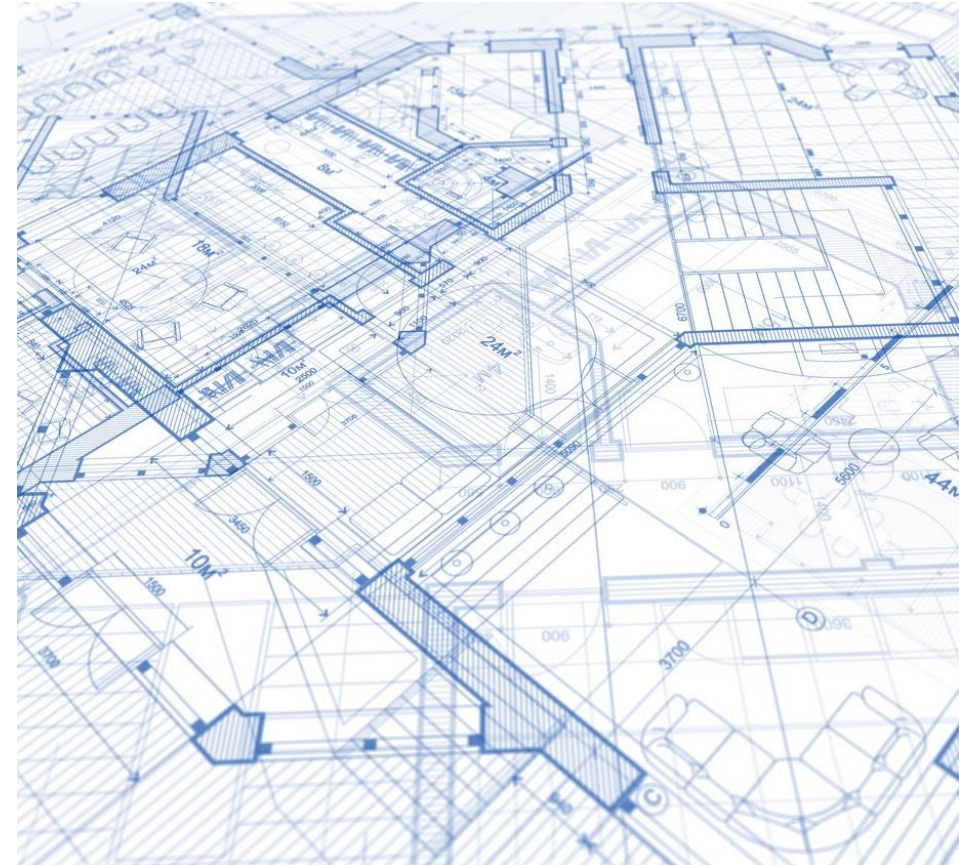
Three ideas make infrastructure behave like software, and they are why an infrastructure-as-code deploy is safe to run a hundred times. Get these and the tool almost does not matter.

The three ideas

- Desired state: declare the end result you want, not the steps to get there
- Idempotence: running it again changes nothing if nothing has changed
- Repeatability: same code produces the same environment every time
- Contrast with a pile of imperative CLI commands that assume a starting point

Key Point:

You already met desired state in Kubernetes. IaC applies the exact same idea to the cloud resources under your app.



BICEP, AND WHY OVER RAW ARM

Azure's native infrastructure format is ARM, Azure Resource Manager, templates written in JSON, which are verbose and painful to write by hand. Bicep is a readable language that compiles straight down to ARM, so you get the same result with far less friction.

What Bicep gives you

- Declarative and Azure-native, with first-class support for every resource
- Transpiles to ARM JSON, so there is nothing new to deploy underneath
- Readable syntax: far less noise than the equivalent JSON
- No state file to manage; Azure itself holds the current state

Key Point:

Bicep is to ARM what a friendly language is to assembly: same result, written by humans instead of fought with.

A FIRST BICEP FILE

The smallest useful Bicep provisions one resource with a couple of parameters and hands back an output. Reading it shows how approachable it is, and it is reviewable in a pull request like any other code.

```
param location string = resourceGroup().location
param env string

resource logs 'Microsoft.OperationalInsights/workspaces@2023-09-01' = {
  name: 'shipit-logs-${env}'
  location: location
  properties: { retentionInDays: 30 }
}

output workspaceId string = logs.id
```

This provisions a Log Analytics workspace, the place App Insights sends its data. The output hands its ID to the next step, and the whole file is reviewable in a PR.

PARAMETERS, VARIABLES, AND OUTPUTS

The same template should provision dev and prod by changing inputs, not by editing the file. Parameters give you that, variables compute derived values, and outputs hand results to the next step.

The three moving parts

- Parameters: what varies per environment, passed in at deploy time
- Variables: computed values you name once and reuse
- Outputs: hand a value to the next step, like a connection string the app needs
- Parameter files per environment keep the template itself untouched

Best practice:

Parameterize what varies and hardcode nothing environment-specific. The same file, different parameters, is the whole point.

MODULES FOR REUSABLE INFRASTRUCTURE

Once you have more than a couple of resources, you factor them into modules for the same reason you write functions: so you reuse and compose instead of copy-pasting. It keeps a large environment readable.

Composing with modules

- A module is just a Bicep file you call with parameters
- Compose a full environment from small, focused modules
- Versioned and shareable across teams and projects
- Keeps a large environment readable instead of one giant file

Key Point:

Modules are to Bicep what functions are to code: the cure for copy-paste, and the way a big environment stays understandable.

POP QUIZ: IDEMPOTENCE

You run your Bicep deploy, then run the exact same deploy again with no changes. What happens to your resources, and why does that matter?



3 MINUTES

POP QUIZ: IDEMPOTENCE

You run your Bicep deploy, then run the exact same deploy again with no changes. What happens to your resources, and why does that matter?

Answer:

Nothing changes. That is idempotence: the deploy reconciles to the declared state, and it already matches.

- It compares desired state to what exists and makes only the needed changes
- No changes needed means no changes made, so re-running is safe
- This is why IaC deploys can run on every pipeline run, unlike a naive script



DEPLOYING BICEP FROM GITHUB ACTIONS

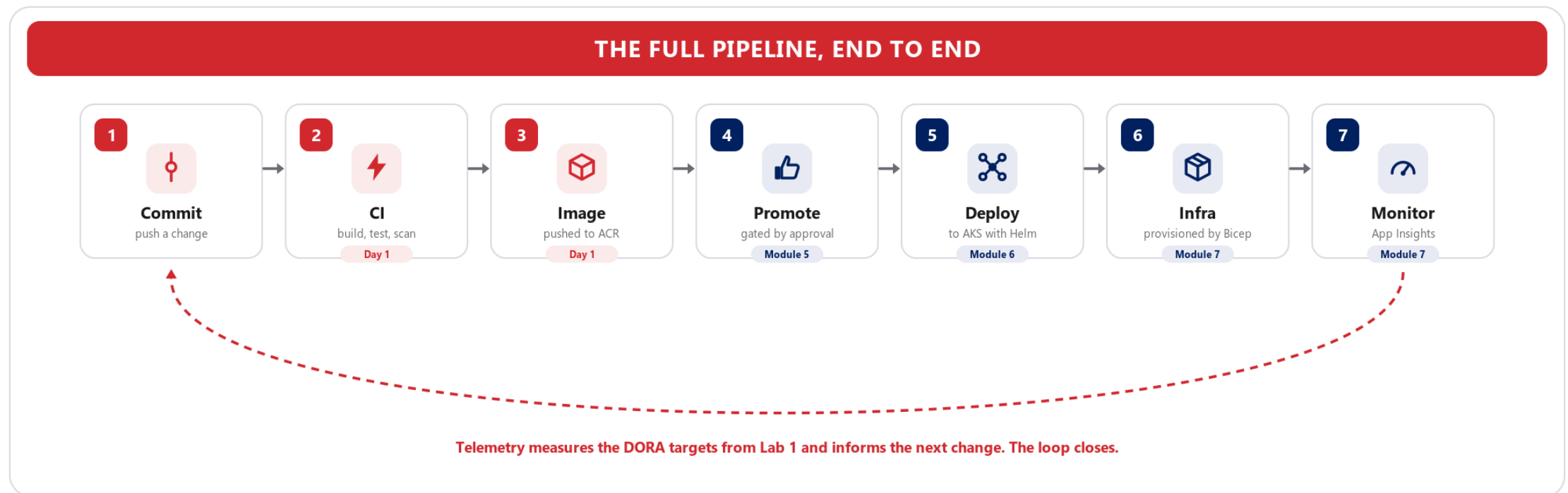
Infrastructure changes belong under the same discipline as code: reviewed in a pull request, gated, and deployed with OIDC and no stored secret. Same login pattern you already know from Module 5.

```
permissions:
  id-token: write
  contents: read
jobs:
  infra:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: azure/login@v3
        with:
          client-id: ${ vars.AZURE_CLIENT_ID }
          tenant-id: ${ vars.AZURE_TENANT_ID }
          subscription-id: ${ vars.AZURE_SUBSCRIPTION_ID }
      - run: |
          az deployment group create -g shipit-rg \
            --template-file infra/main.bicep \
            --parameters env=prod
```

Infrastructure now changes only through a reviewed, audited pipeline, exactly like the app. Same OIDC, same approval gate, no stored secret.

THE FULL PIPELINE, END TO END

Pull the whole two days together into one picture. This is the system you built: one commit flows all the way to a monitored production deployment, and the telemetry feeds back to the metrics we started with.



Commit, CI build and scan, image in ACR, gated promotion, Helm deploy to AKS, Bicep-provisioned infra, and monitoring, whose data closes the loop back to DORA.

WHY YOU MONITOR, FRAMED HONESTLY



Without monitoring, your fastest signal that a release is bad is an angry customer, which is both slow and expensive. Observability turns unknown problems into visible ones you can act on before they spread.

What monitoring buys you

- Detect regressions in minutes, not from a support ticket
- Shrink time-to-restore, which is a DORA metric directly
- Know whether a deploy actually helped or hurt
- Decide with evidence instead of hunches

Key Point:

You cannot improve what you cannot see. Monitoring is what turns the DORA metrics from a goal into a live number.

AZURE MONITOR AND APPLICATION INSIGHTS

Azure Monitor is the umbrella platform; Application Insights is the piece that watches your app specifically, collecting requests, failures, and timings with very little code. It sends its data to a Log Analytics workspace, the one our Bicep just created.

What App Insights collects

- Request rates, response times, and failure rates, automatically
- Exceptions and slow dependencies, so you see what is breaking and why
- Data lands in Log Analytics, queryable and retained
- Metrics, logs, and traces: the count, the record, and the request's journey

Key Point:

App Insights connects to the Log Analytics workspace from our Bicep, so infrastructure as code and monitoring are literally wired together.

INSTRUMENTING SHIPIT

Wiring Application Insights into an ASP.NET Core app is a small change, and doing it makes the status page and API calls show up as live telemetry within minutes.

```
// Program.cs
builder.Services.AddApplicationInsightsTelemetry();

// Connection string comes from configuration, not code:
//   APPLICATIONINSIGHTS_CONNECTION_STRING
// injected into the AKS deployment as a Secret,
// sourced from the Bicep output we created earlier.
```

One line of code plus a connection string. The string comes from the Bicep output, so provisioning and instrumentation stay connected.

DASHBOARDS

A dashboard is the shared view the team actually looks at, so it should answer the questions you ask during and after a deploy at a glance, not show every metric that exists.

A dashboard that earns its place

- Request volume, failure rate, and latency percentiles like P95
- A deploy marker overlay, so you can line up releases against the metrics
- Show what drives a decision, not every metric available
- P95, not just the average: the average hides the slow tail users feel

Best practice:

Watch percentiles, not averages. If the average latency is fine but P95 is terrible, one in twenty users is having a bad time.



ALERTS: TELLING YOU BEFORE THE CUSTOMER DOES



A dashboard only helps when someone is looking at it. An alert reaches out when a metric crosses a line, which is what actually shrinks time-to-restore. The art is alerting on what users feel, not on every internal blip.

Designing alerts that help

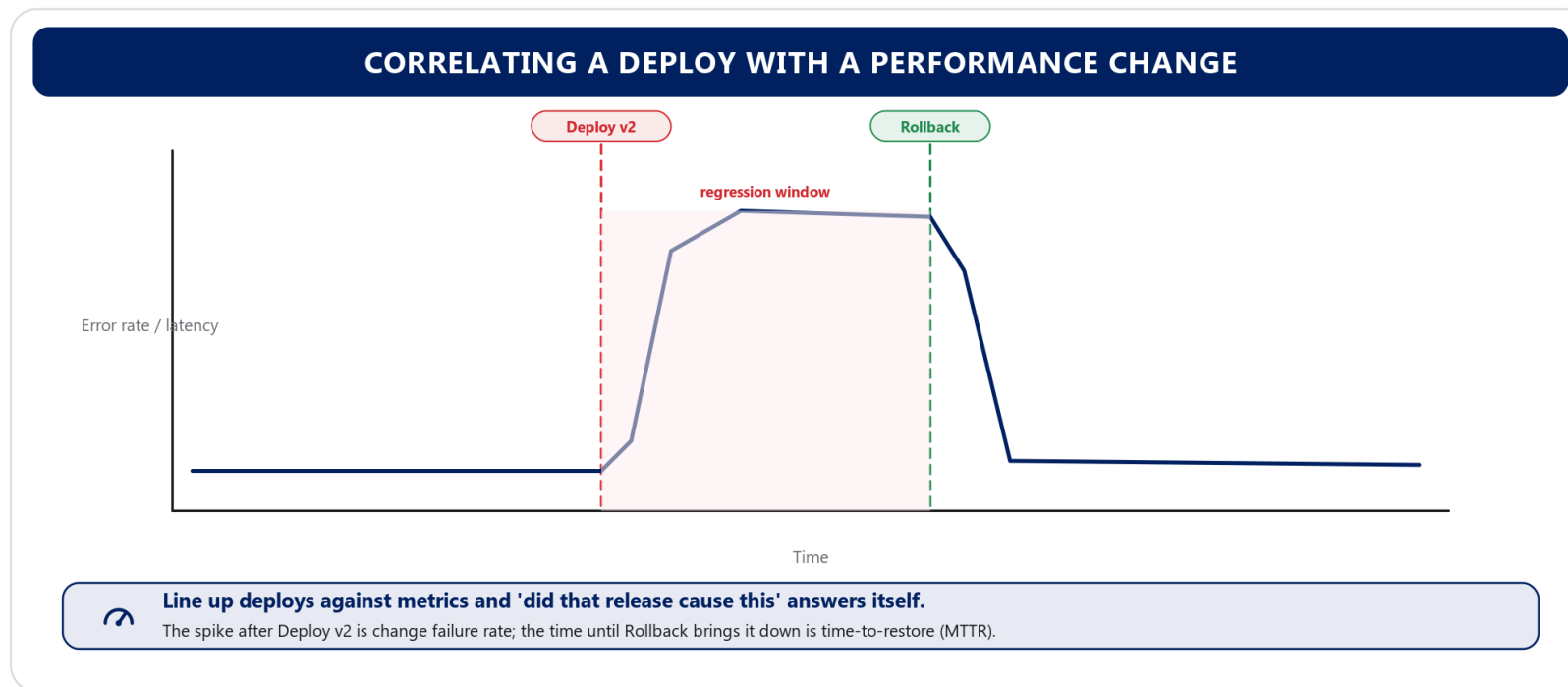
- Metric and log-based alerts on thresholds or dynamic baselines
- Action groups decide who gets paged, and how
- Alert on symptoms users feel: failure rate, latency, availability
- Fewer, sharper alerts beat many noisy ones

Watch out:

Every noisy alert trains people to ignore alerts. The goal is few, high-signal alerts that always mean something.

CORRELATING A DEPLOY WITH A PERFORMANCE CHANGE

The most useful thing monitoring does for delivery is answer did that release cause this, which just requires lining up deploys against metrics on one timeline.



Error rate spikes right after Deploy v2 and returns to baseline after the rollback. That spike is change failure rate; the time to recover is MTTR.

POP QUIZ: ALERT DESIGN

You get paged 40 times a night and you have started ignoring the alerts. Name two changes that would make alerting useful again.



3 MINUTES

POP QUIZ: ALERT DESIGN

You get paged 40 times a night and you have started ignoring the alerts. Name two changes that would make alerting useful again.

Answer:

Tune thresholds or use dynamic baselines so only real problems fire, and alert on symptoms users feel rather than every internal blip.

- Raise or auto-baseline thresholds so normal variation stops paging you
- Alert on failure rate, latency, and availability, not every CPU or memory wiggle
- Fewer, sharper alerts restore trust, so a page means something again



CLOSING THE LOOP, BACK TO DORA

This is where Day 1 and Day 2 meet. The telemetry we now collect measures the exact metrics we set as targets in Lab 1, so improvement stops being a feeling and becomes a number you can move.

Every DORA metric, now measurable

- Deployment frequency: count the pipeline's production runs
- Lead time for changes: commit timestamp to production deploy, from the pipeline
- Change failure rate: failed deploys and post-release alerts over total deploys
- Time to restore: alert-to-recovery, exactly what the rollback chart showed

Key Point:

We opened the course with a scoreboard we could not read. We close it able to measure every number and act on it. That is the DevOps loop.

BICEP BEST PRACTICES

A few habits keep infrastructure code maintainable as it grows, and they mirror the app-code practices from earlier in the course. None of them are new ideas, just applied one layer down.

Habits that scale

- Parameterize what varies; hardcode nothing environment-specific
- Factor into modules for reuse once you have more than a few resources
- Consistent naming and tags, so resources are findable and attributable
- Run what-if before deploy, and review infra changes like app changes

Best practice:

what-if is your plan step: it shows exactly what a deploy would change before it changes anything. Read it every time.

KEY TAKEAWAYS

Infrastructure as code makes your environment repeatable and reviewable; monitoring makes your running app visible. Together they close the DevOps loop, because you can finally measure the DORA metrics you started with.

- IaC gives desired state, idempotence, and repeatability; Bicep is readable ARM
- Infrastructure deploys through the same gated, OIDC pipeline as the app
- Application Insights turns a running app into visible telemetry with one line
- Dashboards and sharp alerts shrink time-to-restore
- Monitoring closes the loop by measuring the DORA metrics from Module 1

LAB 7: PROVISION INFRASTRUCTURE WITH BICEP AND ADD A MONITORING ALERT

Close the loop yourself. You will provision a Log Analytics workspace and Application Insights with Bicep from the pipeline, instrument Shiplt, build a small dashboard, and add an alert that fires on a bad deploy and clears on rollback.

In this lab you will

- Write Bicep for a Log Analytics workspace and App Insights, deployed with OIDC
- Instrument Shiplt and confirm live telemetry from the AKS-hosted app
- Build a dashboard and an alert, then deploy a bad version and watch it fire, roll back, and recover

Hands-on lab:

About 60 minutes. Done when: an alert fires on a deploy-driven change and clears on rollback, and you can state how each DORA metric is measured from this data.



COURSE WRAP-UP: WHAT YOU BUILT AND WHERE TO GO NEXT

Over two days you took one application, Shipt, from a single commit all the way to a monitored production deployment that ships itself, one capability at a time. The tools served one goal: a fast, safe, measurable delivery loop.

What you built, and what is next

- Built: CI, containers, security gates, gated CD, Kubernetes, IaC, and monitoring, on one app
- The through-line: culture and measurement first, tools in service of them
- Next: canary and progressive delivery, policy as code, and GitOps
- Keep using the DORA loop: measure, improve one thing, measure again

Key Point:

You did not just learn seven tools. You built one delivery system and learned the loop that keeps making it better.

Congratulations

You have completed DevOps: CI/CD.



"Develop a passion for learning."